

Lecture 5

Problem Solving as Search

[Adversarial Search]

Resources for this lecture

This lecture covers the following chapters/sections:

- **Chapter 4 (Heuristic Search)** *from* George F. Luger, "Artificial Intelligence: Structures and strategies for complex problem solving," Sixth edition (2009), Pearson Education Limited.

Outline

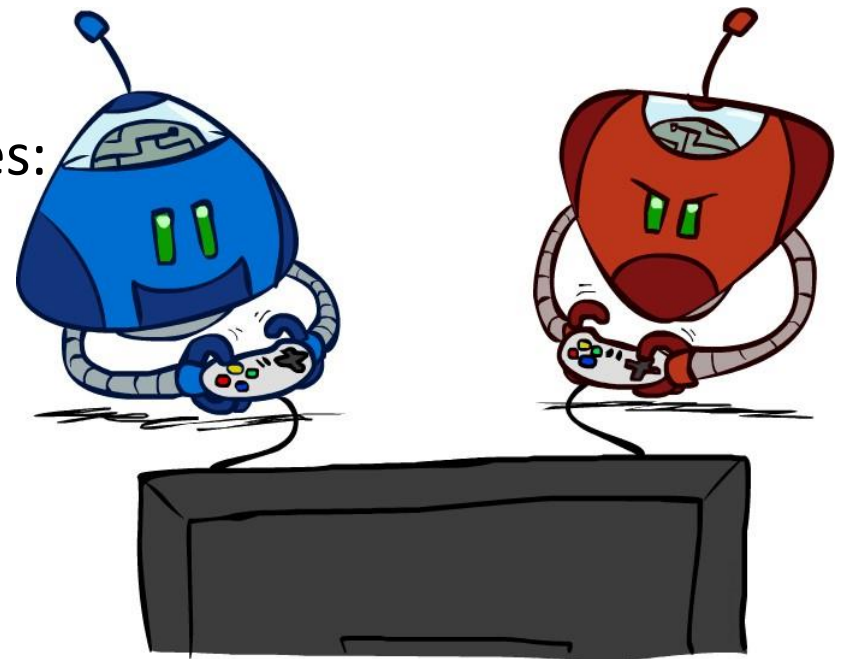
- **Adversarial Search**
(Heuristics in Two-Person Games)
 - **Minimax Procedure**
 - **Exhaustive Minimax for Nim (Splitting Stacks)**
 - **Minimax to a Fixed Ply Depth**
 - **Alpha-Beta Procedure**
 - **Alpha-Beta Example**
- **Search ... What's the Issue?**
 - **Other Search Techniques**
 - **When to Use Search Techniques?**

Adversarial Search (Heuristics in Two-Person Games)

- Two players use and apply the same knowledge of the state space in a consistent effort to win the game.
- Primary difficulty is in accounting for the actions of the opponent.

Two **heuristic pruning** procedures:

- Minimax procedure.
- Alpha-beta procedure.



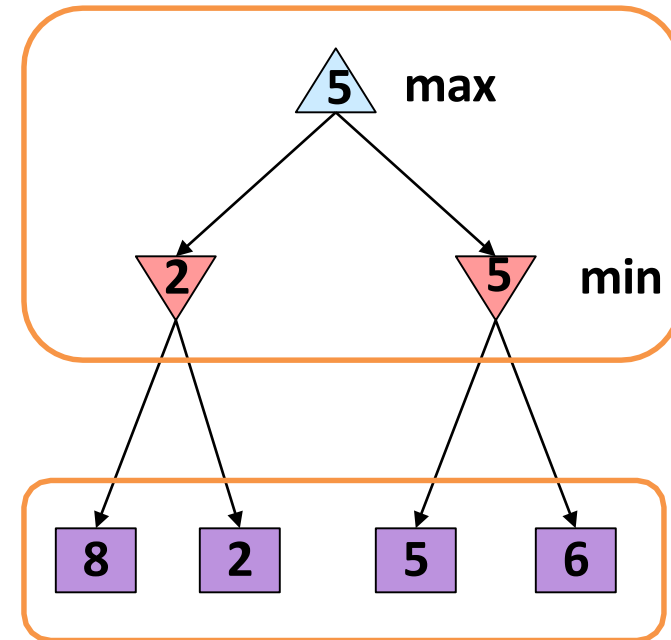
Minimax Procedure

- Opponents are referred to by MIN, and MAX.
- MAX player is trying to Maximize his opportunity to win.
- MIN player is trying to Minimize MAX's opportunity to win.

Implementation:

- Label each level of the search graph with MIN or MAX according to the order of playing.
- Each leaf node is assigned a value of 0 or 1 depending on whether it is a win for MIN, or MAX.
- MAX parent nodes are assigned the maximum of their children.
- MIN parent nodes are assigned the minimum of their children.
- Derived values are used to choose among possible moves.

Minimax values:
Computed Recursively



Terminal values: part of the game

Minimax Tree Example



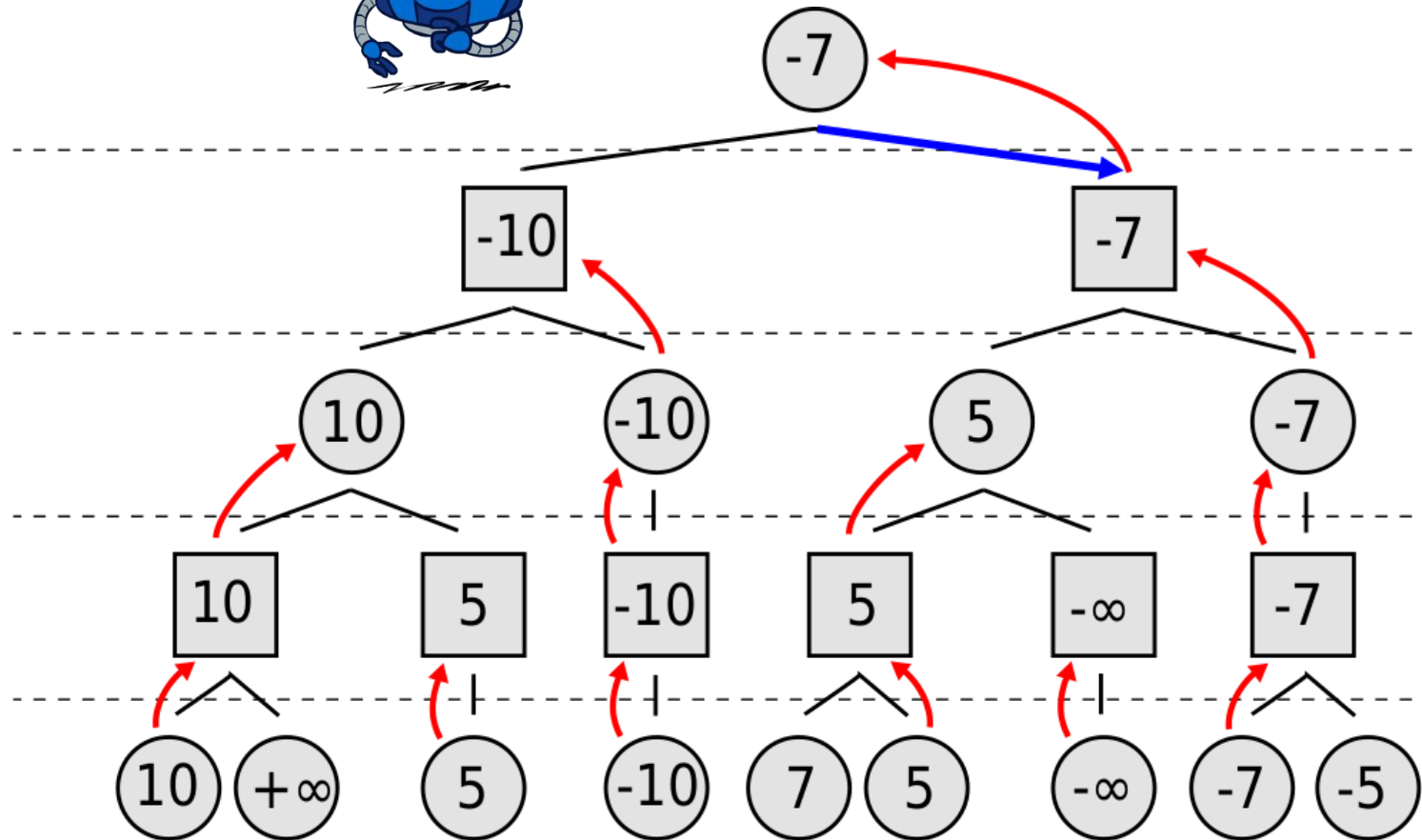
0 (max)

1 (min)

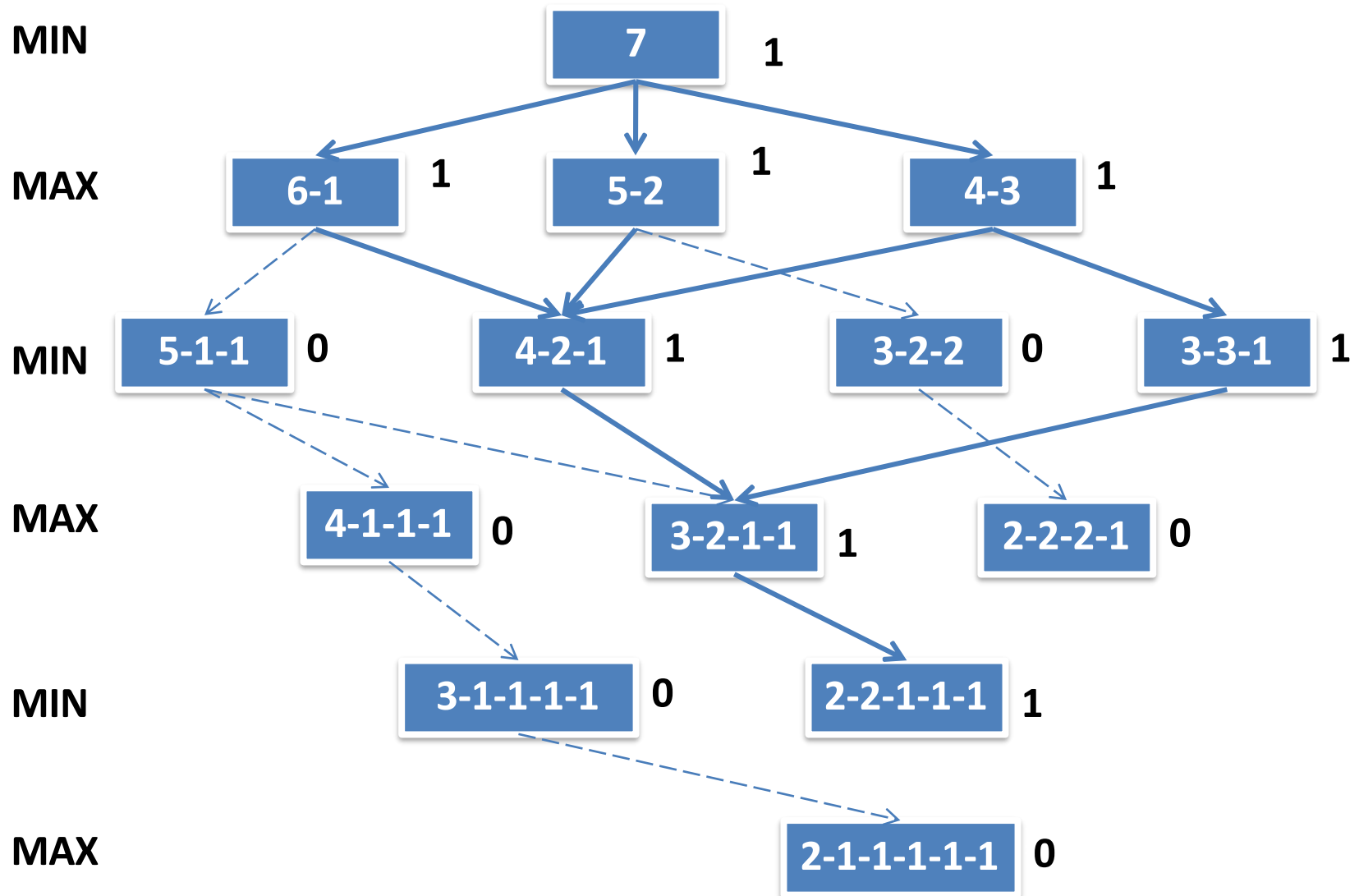
2 (max)

3 (min)

4 (max)

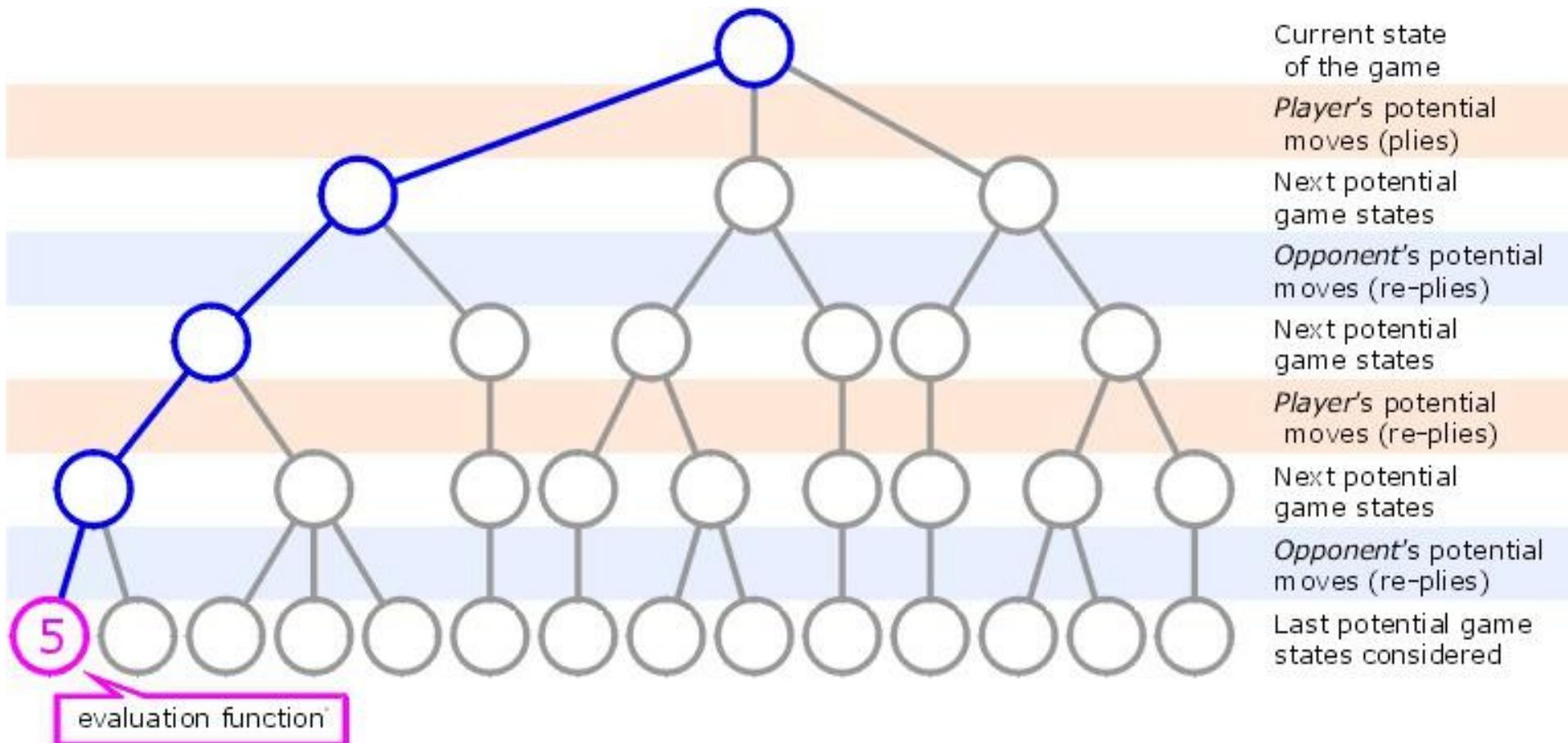


Exhaustive Minimax for Nim (i.e., Splitting Stacks)



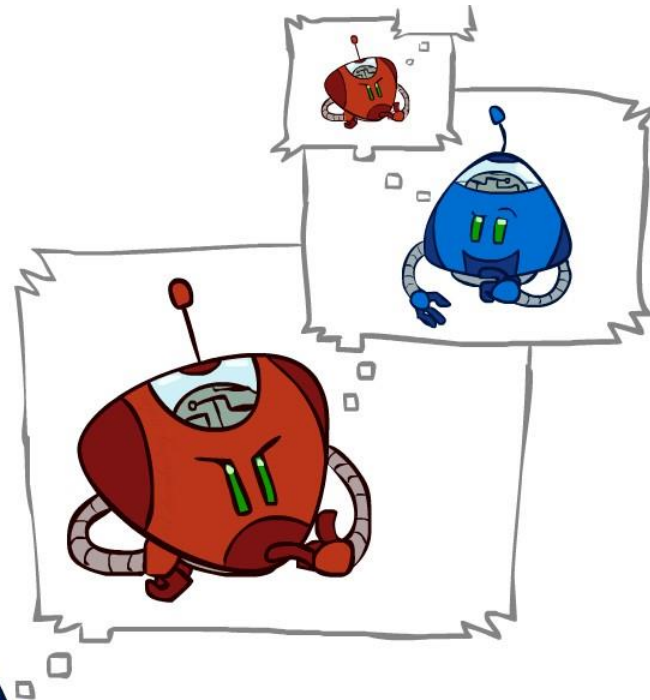
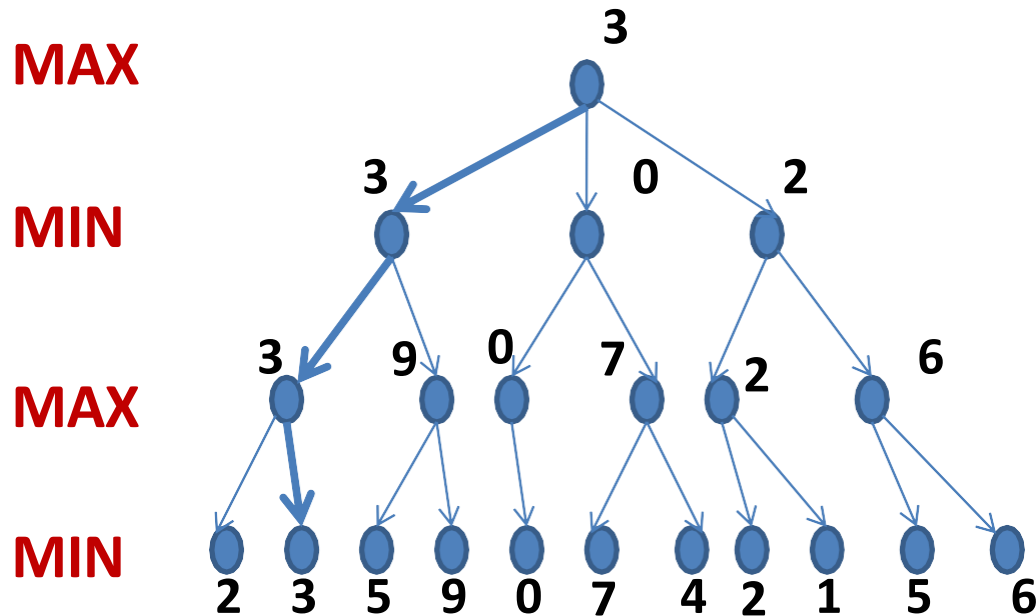
Minimax to a **Fixed Ply Depth**

Example: Minimax on a 2-player game-tree of 4-ply



.. getting this game state's favourability for player.

Minimax to a **Fixed Ply Depth**



Alpha-Beta Pruning Procedure

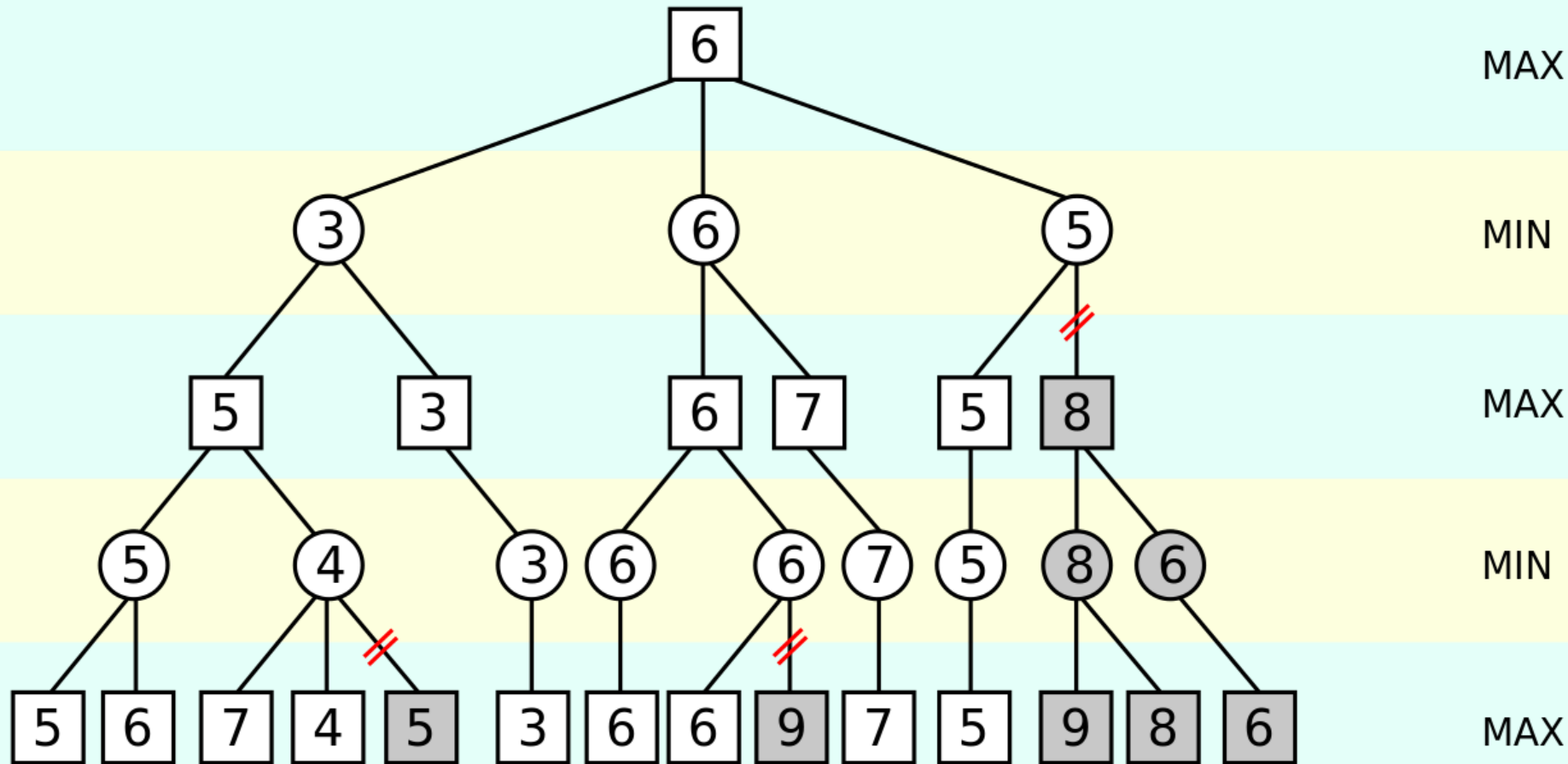
- Alpha-beta pruning is a search algorithm that seeks to **decrease the number of nodes that are evaluated by the minimax algorithm** in its search tree.
- It is an adversarial search algorithm **used commonly for machine playing of two-player games** (Tic-tac-toe, Chess, Go, etc.).
- It stops evaluating a move when at least one possibility has been found that proves the move to be worse than a previously examined move. Such moves need not be evaluated further.
- When applied to a standard minimax tree, it **returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.**

Alpha-Beta Pruning Procedure

α : MAX's best option on path to root
 β : MIN's best option on path to root

- A search space is explored in **depth-first down to the full ply depth**.
- Apply heuristic evaluation to a state and all its siblings.
- For MIN nodes, the maximum is backed up to their parent (*a MAX node*).
- For MAX nodes, the minimum is backed up to their parent (*a MIN node*).
- This value is offered to the grandparent as Beta value for MIN nodes, or Alpha value for MAX nodes.
- **Alpha is a value associated with MAX nodes, that can never decrease.**
- **Beta is a value associated with MIN nodes, that can never increase.**

Alpha-Beta Pruning Procedure..An Illustration



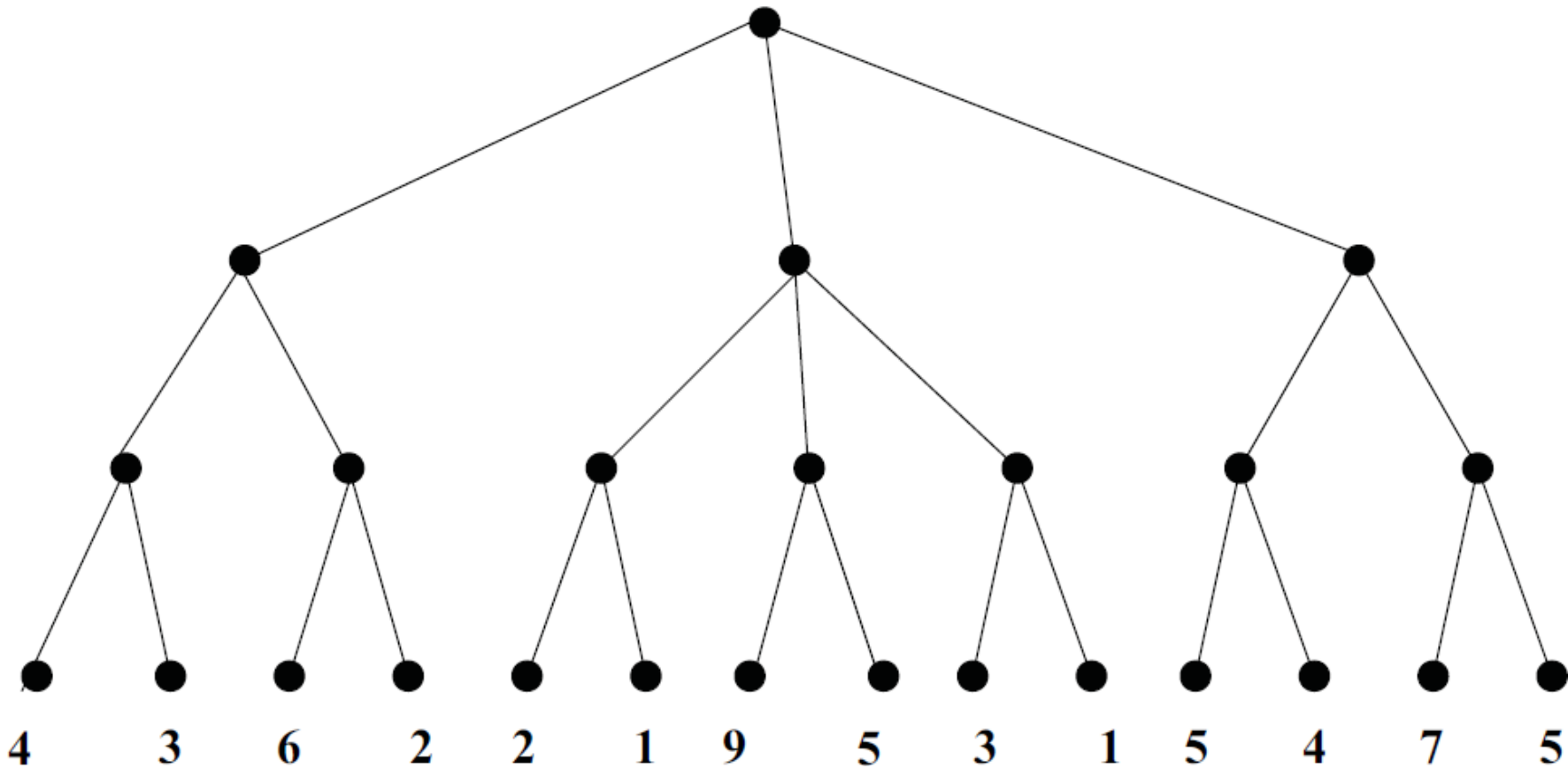
The grayed-out subtrees don't need to be explored (*when moves are evaluated from left to right*), since we know the group of subtrees as a whole yields the value of an equivalent subtree or worse, and as such cannot influence the final result. The max and min levels represent the turn of the player and the adversary, respectively.

Alpha-Beta Pruning Example

MAX

MIN

MAX

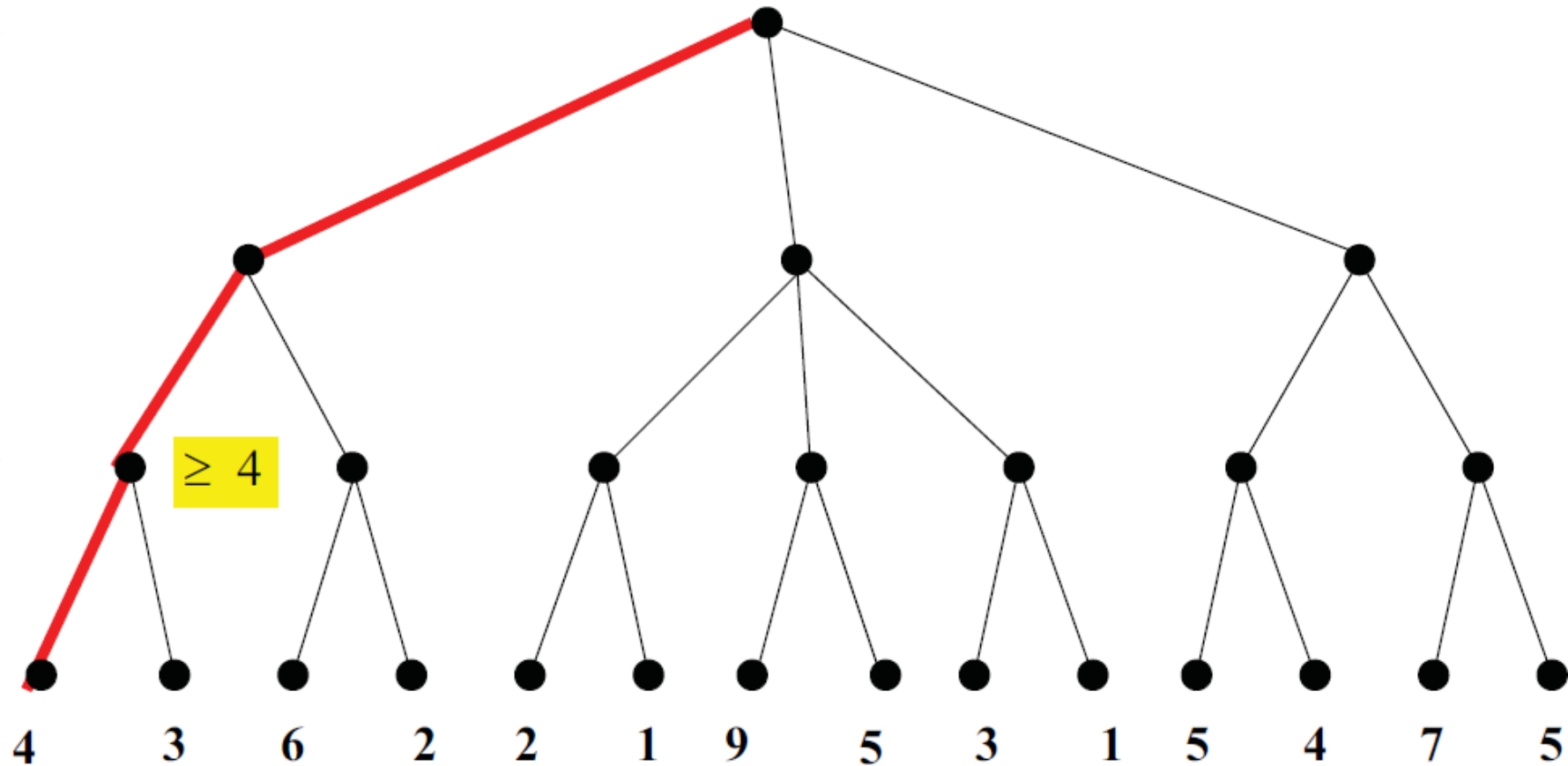


Alpha-Beta Pruning Example

MAX

MIN

MAX

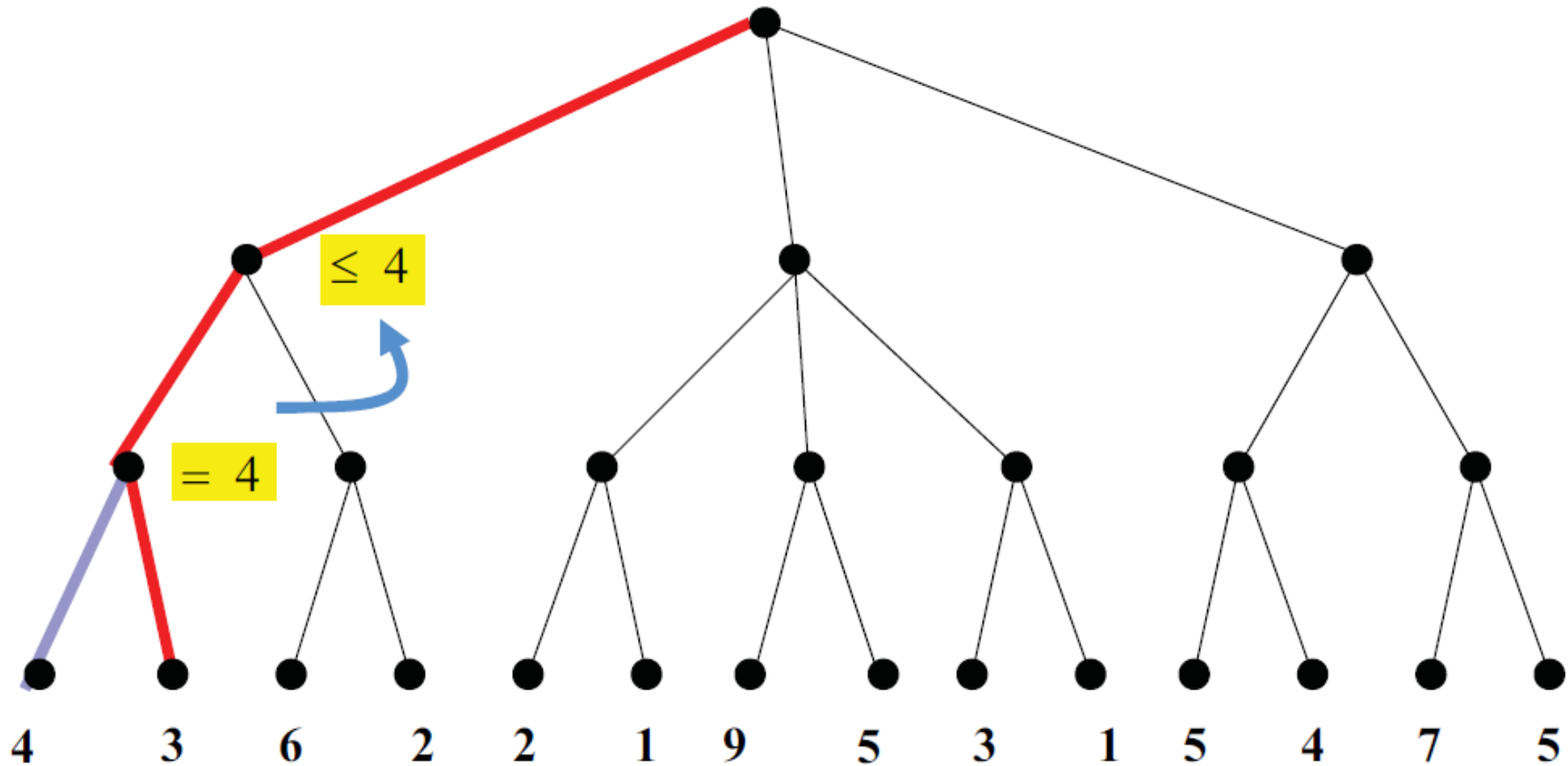


Alpha-Beta Pruning Example

MAX

MIN

MAX

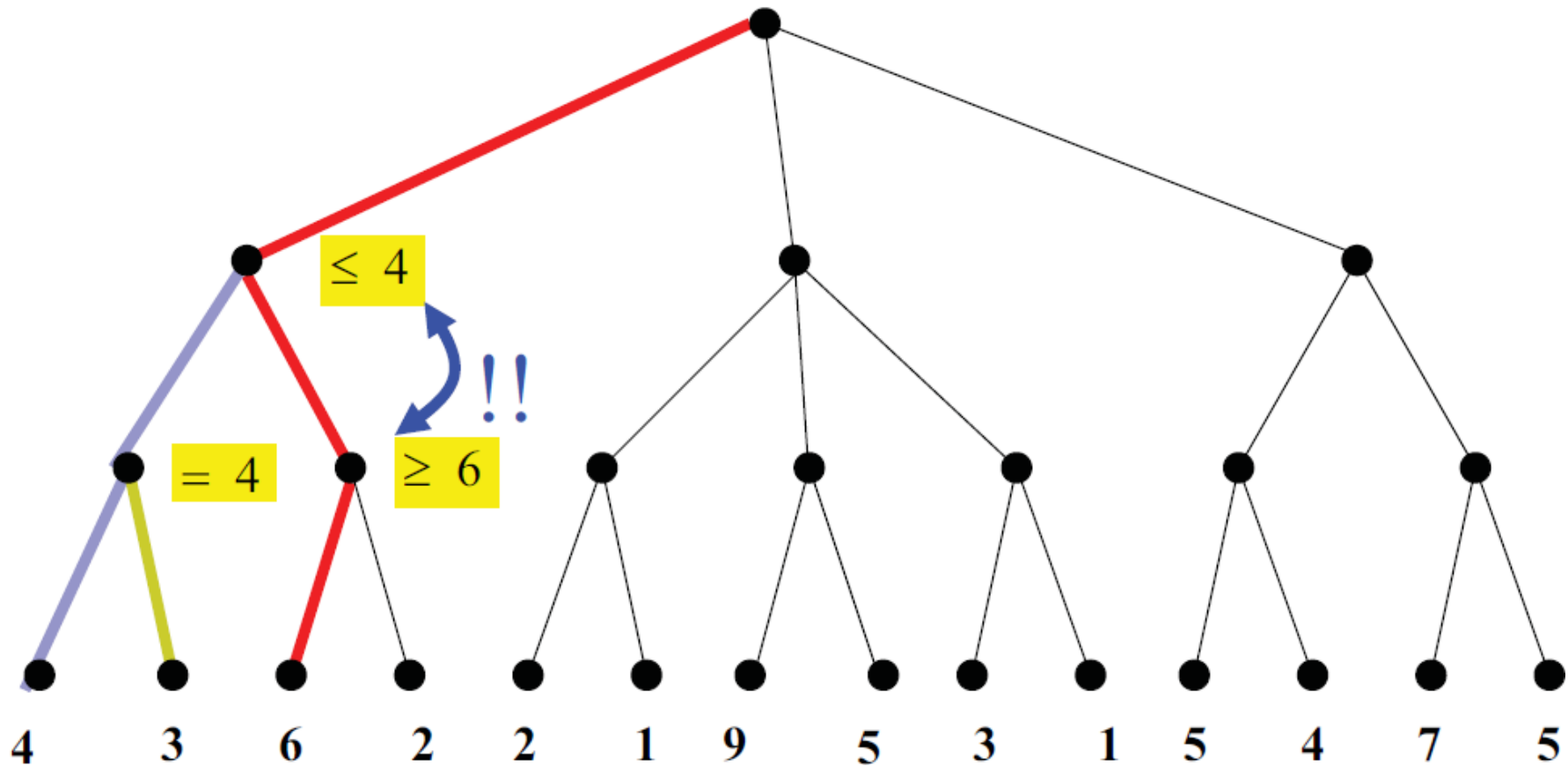


Alpha-Beta Pruning Example

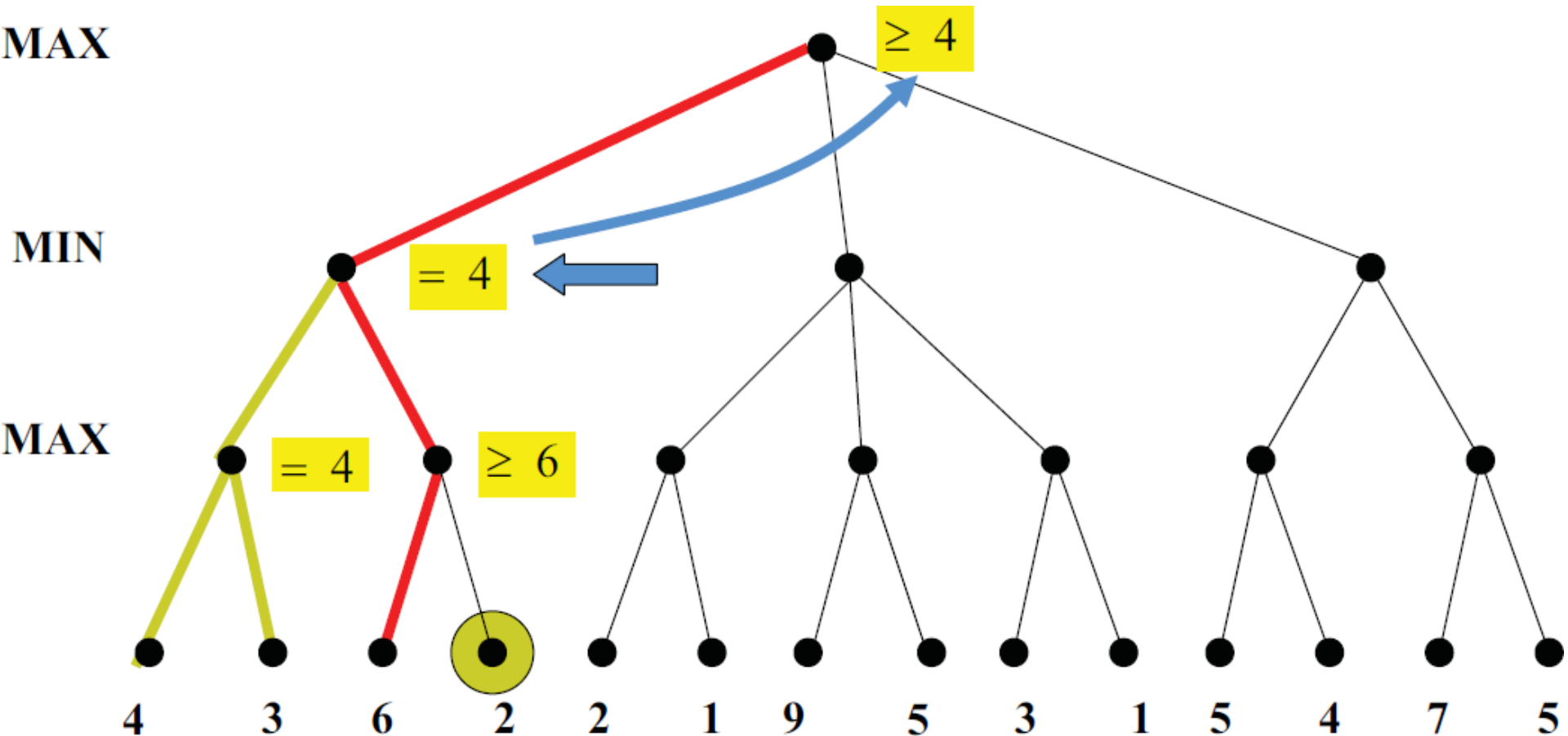
MAX

MIN

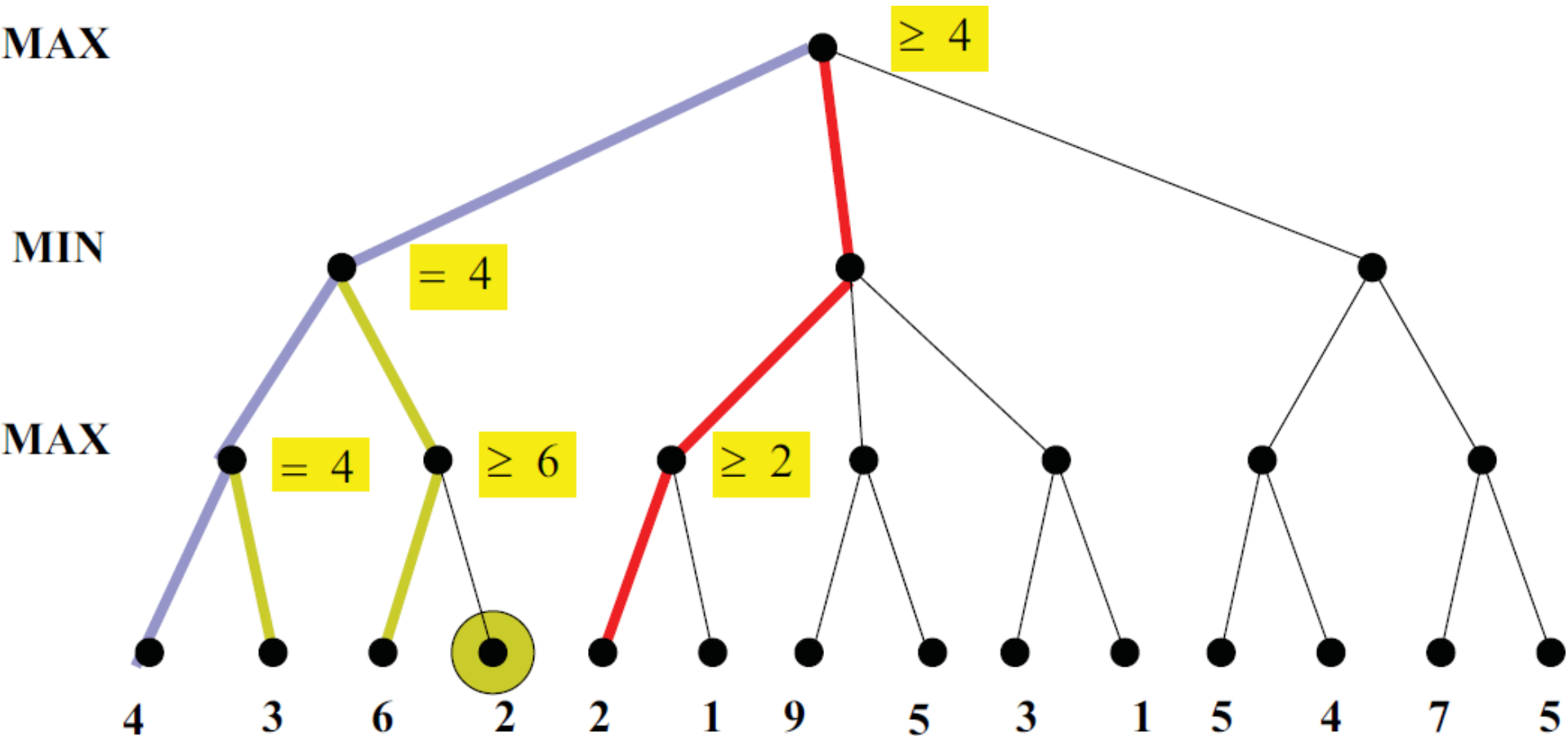
MAX



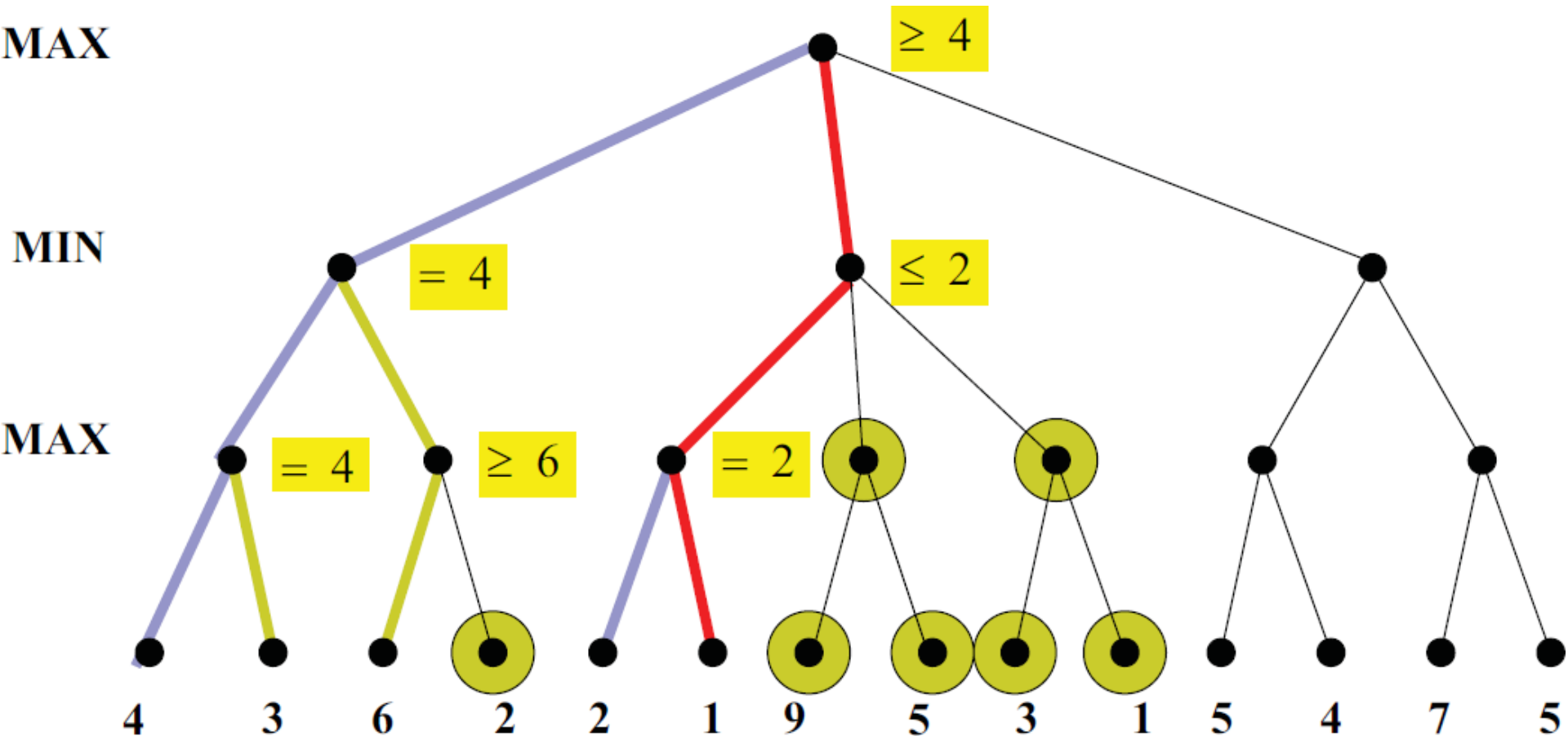
Alpha-Beta Pruning Example



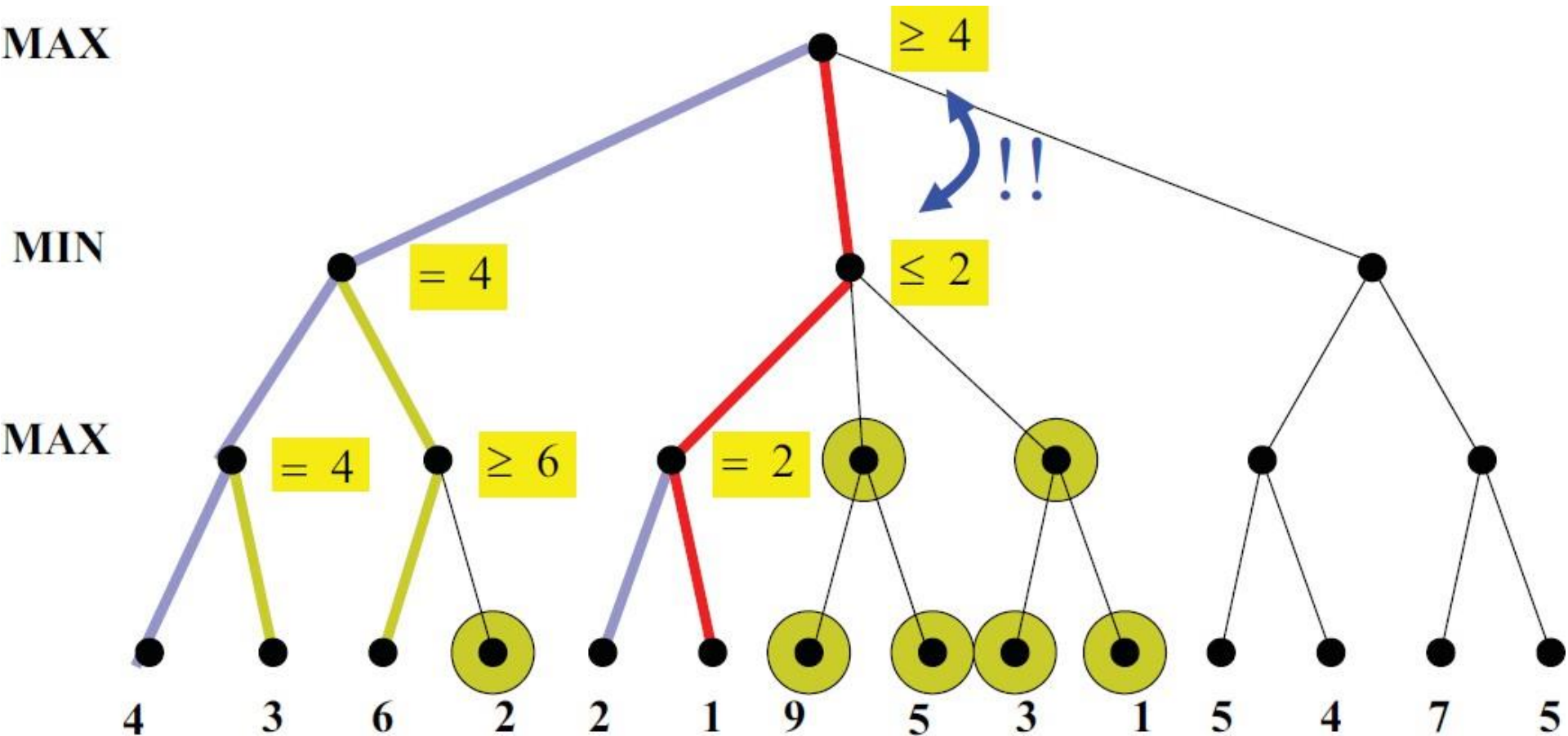
Alpha-Beta Pruning Example



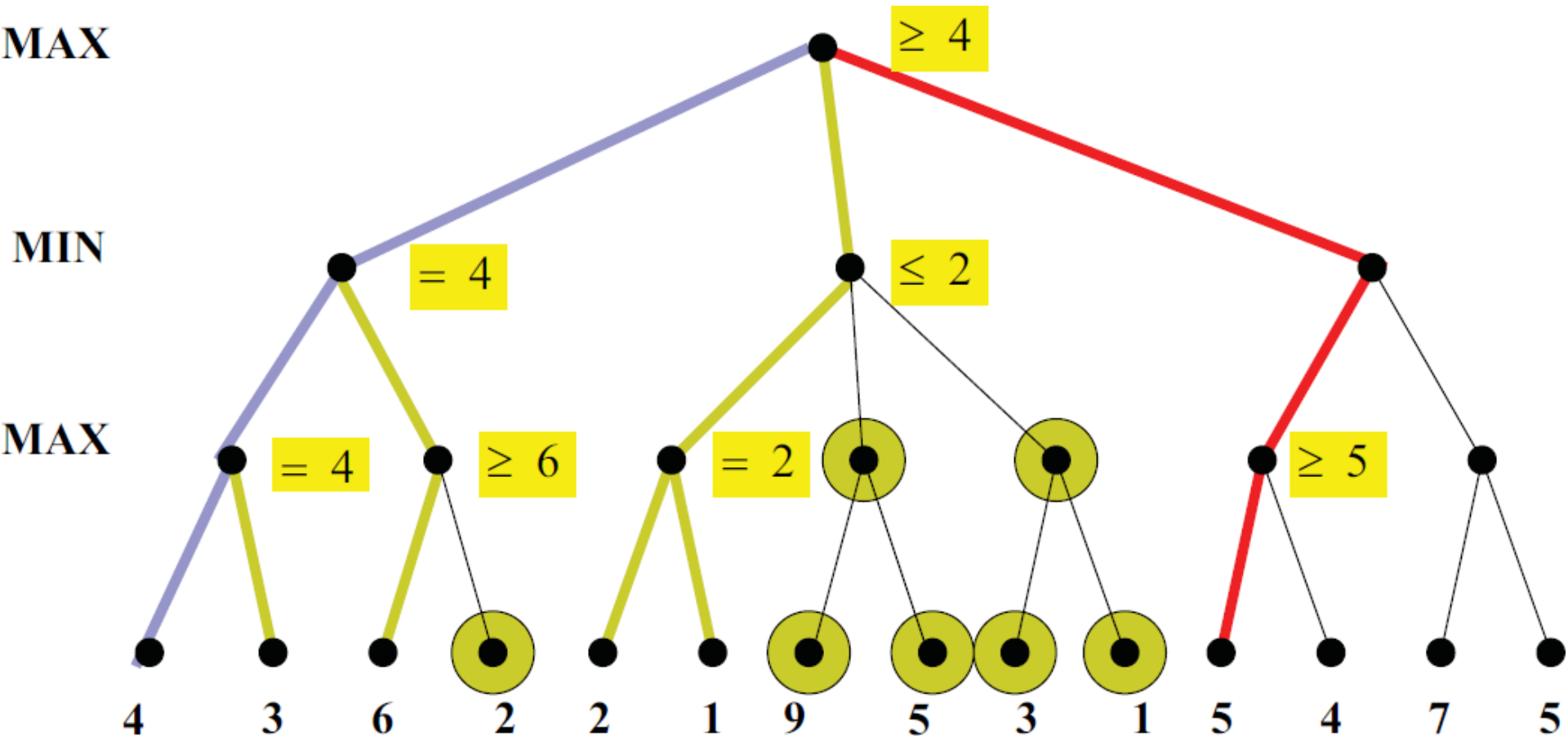
Alpha-Beta Pruning Example



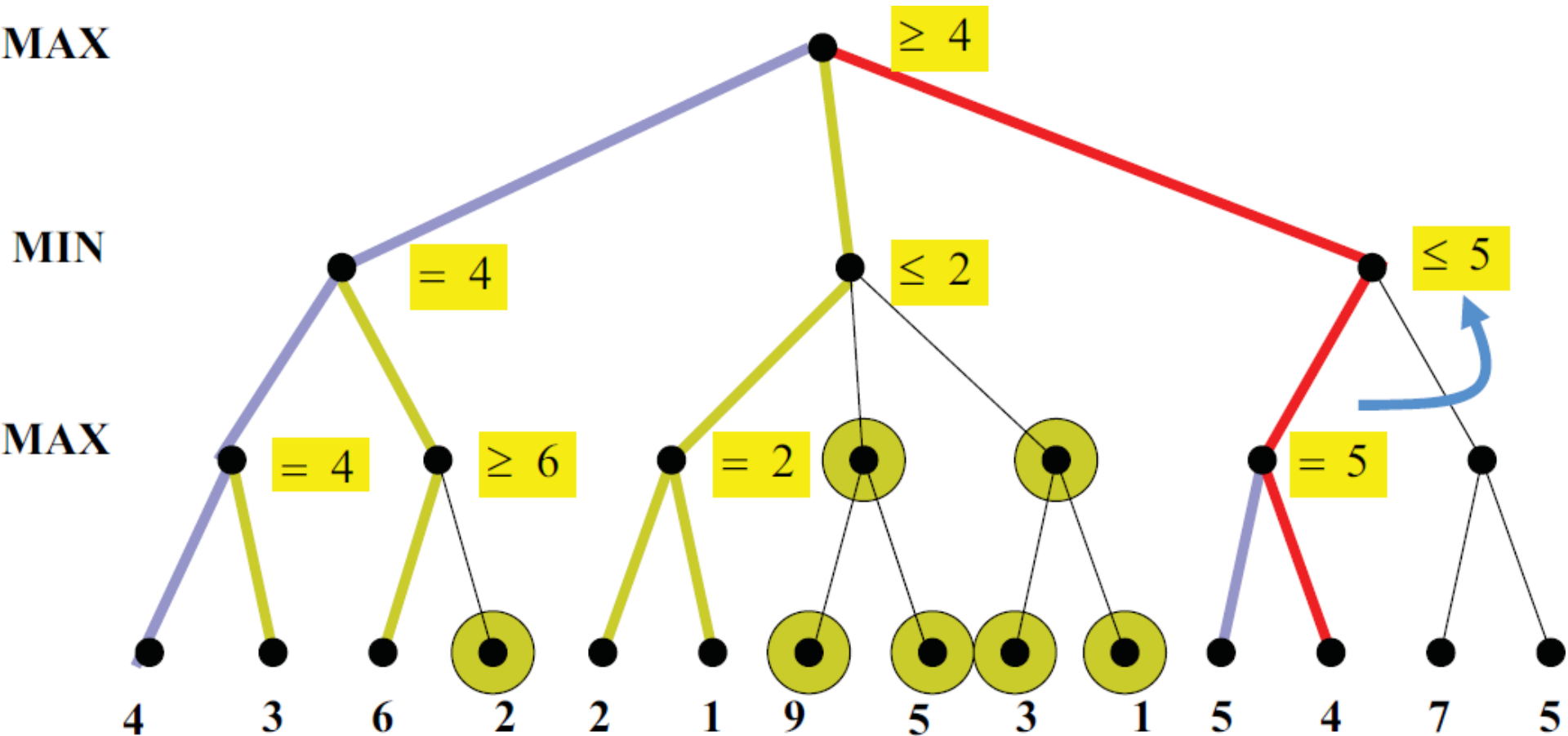
Alpha-Beta Pruning Example



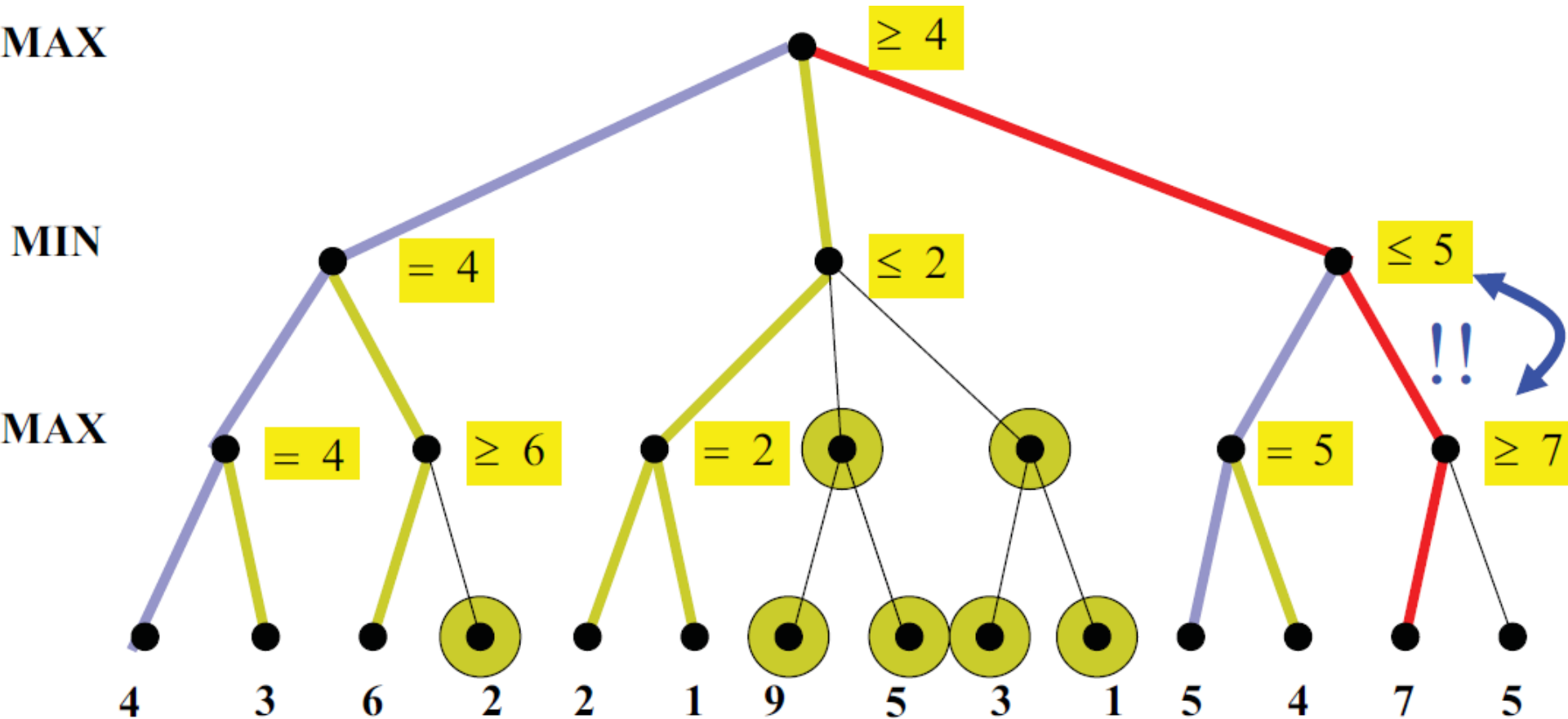
Alpha-Beta Pruning Example



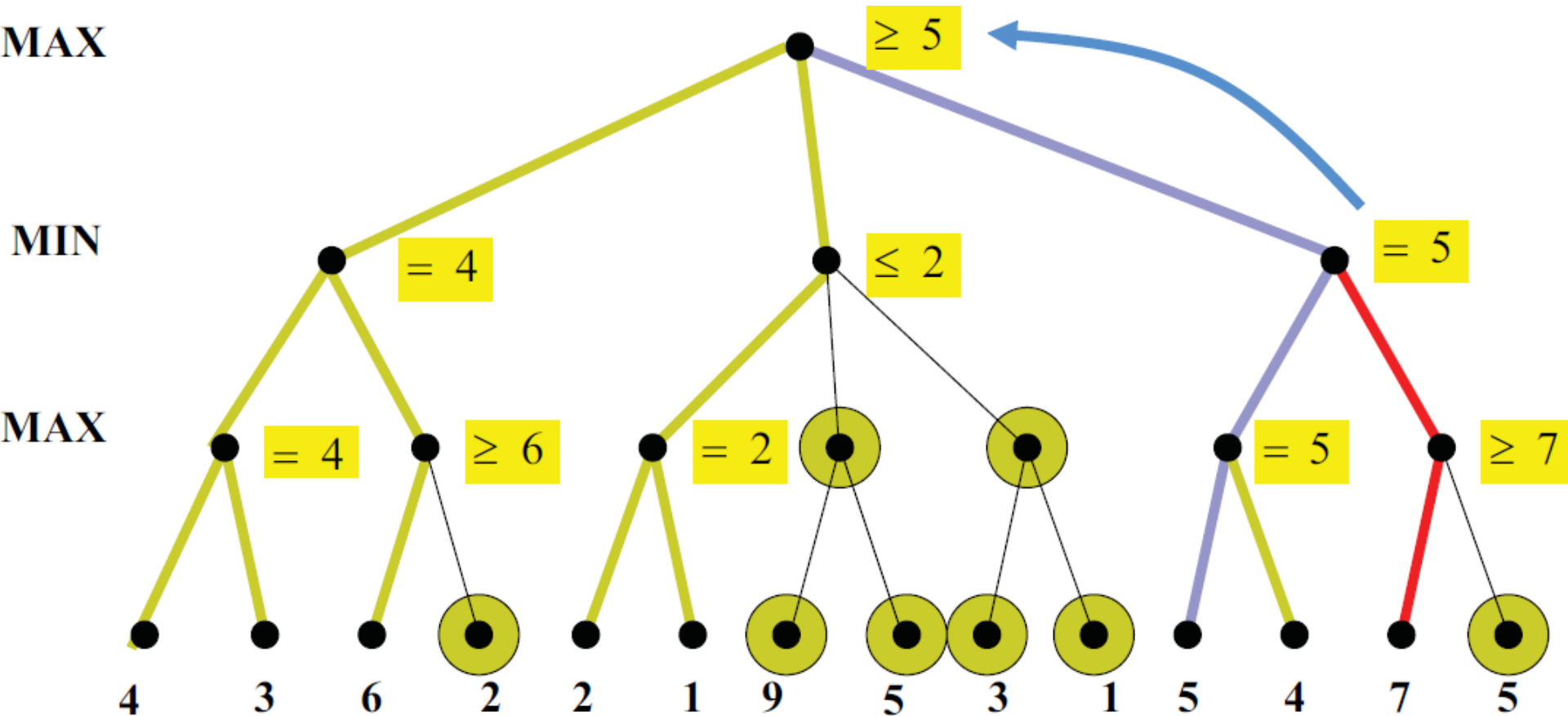
Alpha-Beta Pruning Example



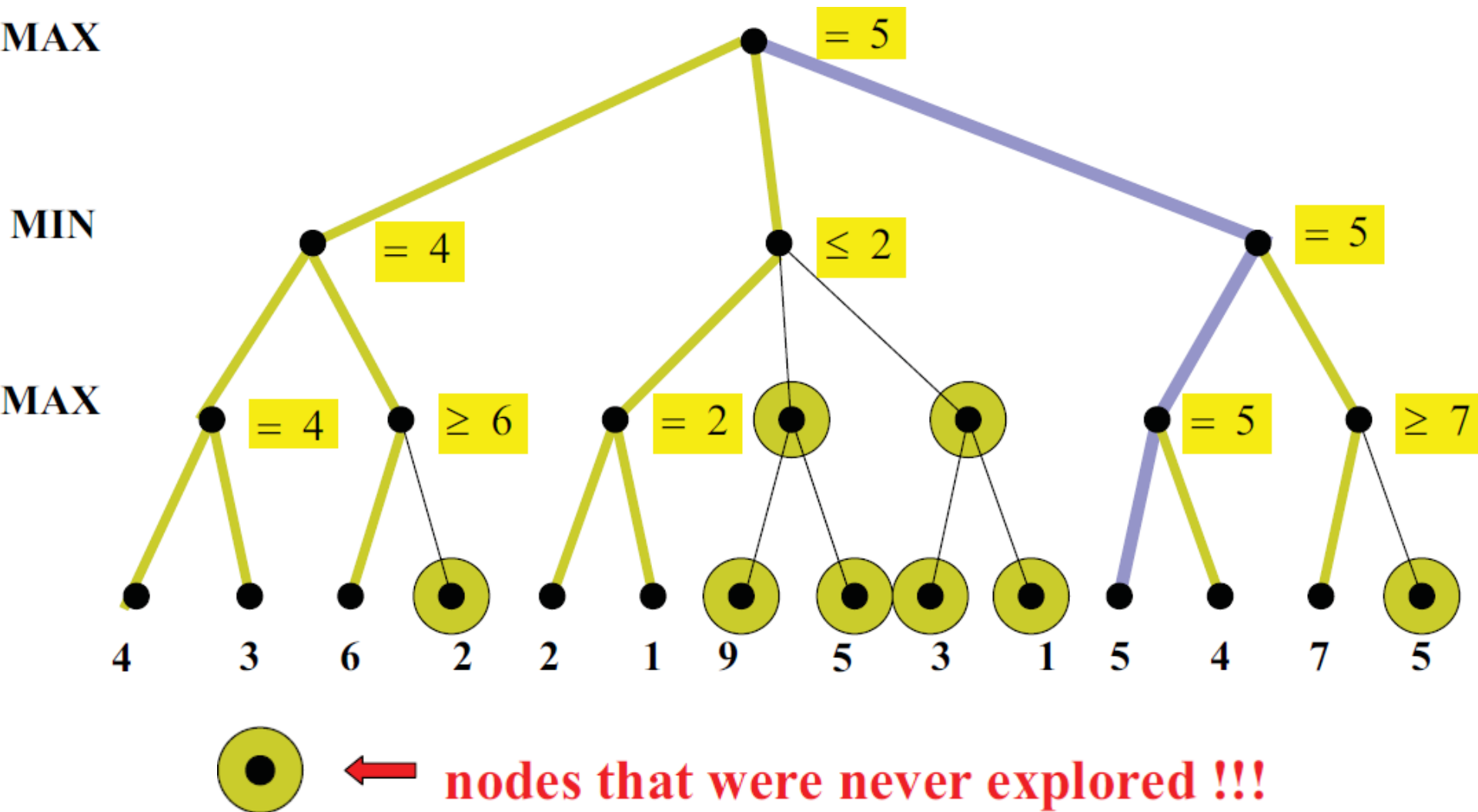
Alpha-Beta Pruning Example



Alpha-Beta Pruning Example



Alpha-Beta Pruning Example



Search.. What's the Issue?

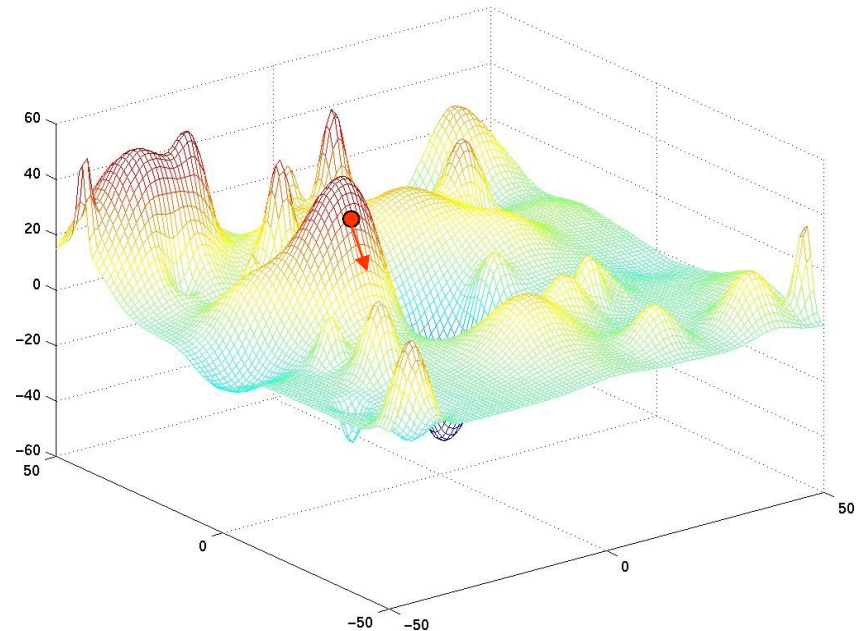
- Search is an iterative **local** procedure
- Good heuristics should provide some **global look-ahead** (at a low computational cost).



Other Search Techniques

Steepest **descent** (~ greedy best-first with no search)

→ may get stuck into local **minimum**



Other Search Techniques

Steepest descent (~ greedy best-first with no search)

→ may get stuck into local minimum

- **Simulated annealing**
- **Genetic algorithms**

When to Use Search Techniques?



The search space is small, *and*

There is no other available techniques,
or

It is not worth the effort to develop a
more efficient technique



The search space is large, *and*

There is no other available techniques,
and

There exist “good” heuristics



Thanks! ...
Questions?